TRABAJO PRACTICO

CLASIFICACIÓN DE FISURAS EN PAVIMENTOS

Redes Neuronales Convolucionales, Transfer Learning y MobileNetV2

Alumno: Diego Alberto Di Stefano

Legajo: 0134844

Materia: Redes Neuronales y Deep Learning

Profesor: Alan Dreszman

Entorno de desarrollo: Google Colab - Python

Fecha de entrega: 9 de octubre de 2026

Índice

Resumen ejecutivo 3
1. Introducción 4
2. Objetivos del proyecto 4
2.1 Objetivo general 4
2.2 Objetivos específicos 4
3. Fundamentos conceptuales 4
3.1 Redes Neuronales Convolucionales 4
3.2 Transfer Learning 5
3.3 MobileNetV2 5
4. Entorno y tecnologías utilizadas 5
5. Dataset y clases analizadas 6
5.1 Carga del dataset 6
5.2 Clases seleccionadas 6
5.3 Exploración y distribución 6
6. Conversión y preparación del dataset 7
6.1 Flujo de procesamiento 7
6.2 División estratificada 8
7. Preprocesamiento y aumento de datos 8
7.1 Redimensionamiento y tamaño de lote 8
7.2 Normalización 8
7.3 Data Augmentation 8
7.4 Generadores de datos 9
8. Arquitectura del modelo 9
9. Compilación y estrategia de entrenamiento 10
9.1 Compilación inicial 10
9.2 Callbacks 10
9.3 Entrenamiento inicial 10
10. Fine tuning 10
11. CNN desde cero 11
11.1 Motivos de fluctuación en entrenamiento y validación 11
12. Evaluación del modelo 12
12.1 Resultados globales 12
12.2 Resultados por clase 12
12.3 Matriz de confusión 13
12.4 Curvas de entrenamiento 13
13. Predicción sobre imágenes nuevas 14

14. Aplicación en empresas constructoras y viales 14
15. Limitaciones identificadas 14
16. Mejoras futuras 15
17. Conclusiones 15
18. Síntesis técnica del experimento 15

Resumen ejecutivo

El trabajo desarrolla un sistema de Inteligencia Artificial orientado a la clasificación automática de fisuras en pavimentos. La solución utiliza visión por computadora, Redes Neuronales Convolucionales y Transfer Learning con MobileNetV2, una arquitectura preentrenada en ImageNet y optimizada para lograr una relación favorable entre precisión y eficiencia computacional.

El dataset utilizado fue Road Damage Dataset (CC0), descargado desde Kaggle y cargado en Google Colab mediante Google Drive. A partir de 3.321 imágenes originales y de sus anotaciones en formato YOLO, se localizaron las regiones dañadas mediante bounding boxes y se generaron 4.843 recortes individuales. Estos recortes fueron organizados en tres categorías: fisura longitudinal, fisura transversal y fisura tipo piel de cocodrilo.

El conjunto procesado se dividió de forma estratificada en 70 % para entrenamiento, 15 % para validación y 15 % para prueba. Las imágenes se redimensionaron a 224 × 224 píxeles, se normalizaron con los parámetros de ImageNet y se aplicó aumento de datos mediante Albumentations. El entrenamiento se realizó en dos fases: una etapa inicial con MobileNetV2 congelada y una etapa posterior de fine tuning sobre las capas superiores. Como referencia experimental se implementa una CNN de cuatro bloques convolucionales, inicializada sin pesos preentrenados y entrenada con las mismas particiones de datos.

El modelo final obtuvo una accuracy de 84,87 % y una pérdida de 0,3764 sobre 727 ejemplos de prueba. El mejor desempeño se observó en la clase Longitudinal Crack, con recall de 0,94 y F1-score de 0,88. La principal dificultad apareció en Transverse Crack, cuyo recall fue 0,55, indicando que una parte importante de estas fisuras fue confundida principalmente con fisuras longitudinales.

Resultado principal: Accuracy final de 84,87 % sobre el conjunto de prueba, con 617 clasificaciones correctas de 727 imágenes.

1. Introducción

La conservación de las redes viales exige inspecciones periódicas capaces de identificar daños antes de que evolucionen hacia fallas más graves. Las fisuras son uno de los indicadores más frecuentes del deterioro del pavimento y su clasificación permite orientar las tareas de mantenimiento, estimar prioridades y seleccionar tratamientos adecuados.

La inspección manual presenta limitaciones: requiere tiempo, depende de la experiencia del evaluador, puede ser subjetiva y resulta costosa cuando deben analizarse grandes extensiones. En este contexto, la visión por computadora y el aprendizaje profundo ofrecen la posibilidad de automatizar parte del proceso mediante el análisis de imágenes capturadas desde vehículos, cámaras, teléfonos, drones u otros dispositivos.

El proyecto propone un clasificador de imágenes centrado en tres tipos de fisuras. El sistema no detecta por sí solo la ubicación de los daños en una imagen completa: utiliza las anotaciones YOLO del dataset para recortar cada fisura y luego clasifica el recorte. Esta decisión permite transformar un problema de detección de objetos en un problema de clasificación multiclase.

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar y evaluar un modelo de Inteligencia Artificial capaz de clasificar automáticamente diferentes tipos de fisuras en pavimentos a partir de imágenes, utilizando una CNN preentrenada y la técnica de Transfer Learning.

2.2 Objetivos específicos

- Explorar el Road Damage Dataset y comprender su estructura de imágenes y anotaciones YOLO.
- Seleccionar tres clases relevantes de daños: Longitudinal Crack, Transverse Crack y Alligator Crack.
- Convertir las anotaciones de detección en recortes individuales aptos para clasificación.
- Dividir los datos de manera estratificada en entrenamiento, validación y prueba.
- Aplicar redimensionamiento, normalización y aumento de datos.
- Adaptar MobileNetV2 mediante Transfer Learning y capas de clasificación personalizadas.
- Realizar un entrenamiento inicial y una segunda fase de fine tuning.
- Realizar CNN de referencia sin preentrenamiento.
- Evaluar el desempeño mediante accuracy, pérdida, precisión, recall, F1-score y matriz de confusión.
- Preparar un mecanismo para clasificar imágenes nuevas y medir el tiempo de inferencia.

3. Fundamentos conceptuales

3.1 Redes Neuronales Convolucionales

Una Red Neuronal Convolucional o CNN es una arquitectura de aprendizaje profundo especialmente adecuada para imágenes. Sus capas convolucionales aprenden filtros capaces de reconocer patrones locales. En las primeras etapas se identifican bordes, contrastes y texturas; en etapas posteriores se combinan esas señales para reconocer formas y estructuras más complejas.

Esta capacidad de aprendizaje jerárquico evita definir manualmente las características visuales de cada fisura. El modelo aprende directamente de los ejemplos y ajusta sus parámetros para diferenciar los patrones longitudinales, transversales y reticulados propios de la piel de cocodrilo.

3.2 Transfer Learning

Transfer Learning consiste en reutilizar un modelo entrenado previamente en una tarea amplia como punto de partida para una nueva tarea específica. En este proyecto se aprovechan los pesos aprendidos por MobileNetV2 sobre ImageNet. Aunque ImageNet no está especializado en pavimentos, sus capas ya reconocen características visuales generales como bordes, contornos, texturas y formas.

La ventaja principal es que no resulta necesario entrenar una red profunda desde cero. Esto reduce el volumen de datos requerido, disminuye el costo computacional, acelera la convergencia y suele mejorar el rendimiento cuando el dataset específico es limitado.

3.3 MobileNetV2

MobileNetV2 es una arquitectura diseñada por Google para aplicaciones móviles y dispositivos con recursos limitados. Utiliza convoluciones separables en profundidad y bloques de cuello de botella invertidos con conexiones residuales. Estas decisiones reducen la cantidad de operaciones y parámetros sin perder demasiada capacidad de representación.

La elección fue adecuada para el TP porque combina disponibilidad en TensorFlow/Keras, pesos preentrenados, velocidad de entrenamiento, inferencia eficiente y un tamaño de modelo compatible con una futura implementación en campo o en dispositivos edge.

4. Entorno y tecnologías utilizadas

El desarrollo se ejecutó en Google Colab con Python 3. Este entorno permitió utilizar recursos de cómputo en la nube, acceder al dataset almacenado en Google Drive y trabajar con el ecosistema de aprendizaje automático de TensorFlow.

Tecnología	Uso en el proyecto
TensorFlow y Keras	Construcción, compilación, entrenamiento, fine tuning, almacenamiento y carga del modelo.
MobileNetV2	Modelo base preentrenado utilizado como extractor de características.
OpenCV	Lectura de imágenes, conversión de color, recorte mediante bounding boxes, dibujo y guardado.
Albumentations	Aumento de datos y normalización de imágenes.
NumPy	Manejo de arreglos numéricos, probabilidades y selección de clases.
Pandas	Organización de anotaciones, rutas, clases y particiones mediante DataFrames y archivos CSV.
Scikit-learn	División estratificada, reporte de clasificación y matriz de confusión.
Matplotlib y Seaborn	Visualización de ejemplos, distribución de clases, curvas y matriz de confusión.

Tecnología	Uso en el proyecto
tqdm	Barras de progreso para procesos extensos.
Google Drive / Colab Files	Carga del dataset y prueba con imágenes nuevas.

5. Dataset y clases analizadas

Se utilizó el Road Damage Dataset (CC0) disponible en Kaggle. El dataset contiene fotografías de pavimentos y archivos de anotación asociados en formato YOLO. En este formato, cada daño se representa mediante un identificador de clase y un bounding box normalizado.

Link Dataset: https://www.kaggle.com/datasets/alvarobasily/road-damage

5.1 Carga del dataset

El archivo comprimido fue almacenado en Google Drive, montado desde Colab y descomprimido en el entorno de ejecución. Se verificó el contenido del directorio y trabajó con pares de archivos de imagen JPEG y archivos TXT de anotaciones.

5.2 Clases seleccionadas

ID	Clase	Descripción visual general
0	Longitudinal Crack	Fisura predominante en la dirección longitudinal de la vía.
1	Transverse Crack	Fisura que atraviesa el pavimento en dirección transversal.
2	Alligator Crack	Red de fisuras interconectadas semejante a la piel de un cocodrilo.

En la notebook se dejó prevista como mejora futura la incorporación de una clase “Sin fisuras”, necesaria para diferenciar daños reales de imágenes sin deterioro relevante.

5.3 Exploración y distribución

Se localizaron 3.321 imágenes originales. Como una misma imagen puede contener más de una anotación, la cantidad de instancias de fisuras fue superior al número de imágenes. Se identificaron 4.843 instancias distribuidas de la siguiente manera:

Clase	Instancias	Participación aproximada
Longitudinal Crack	2.657	54,9 %
Transverse Crack	964	19,9 %
Alligator Crack	1.222	25,2 %
Total	4.843	100 %

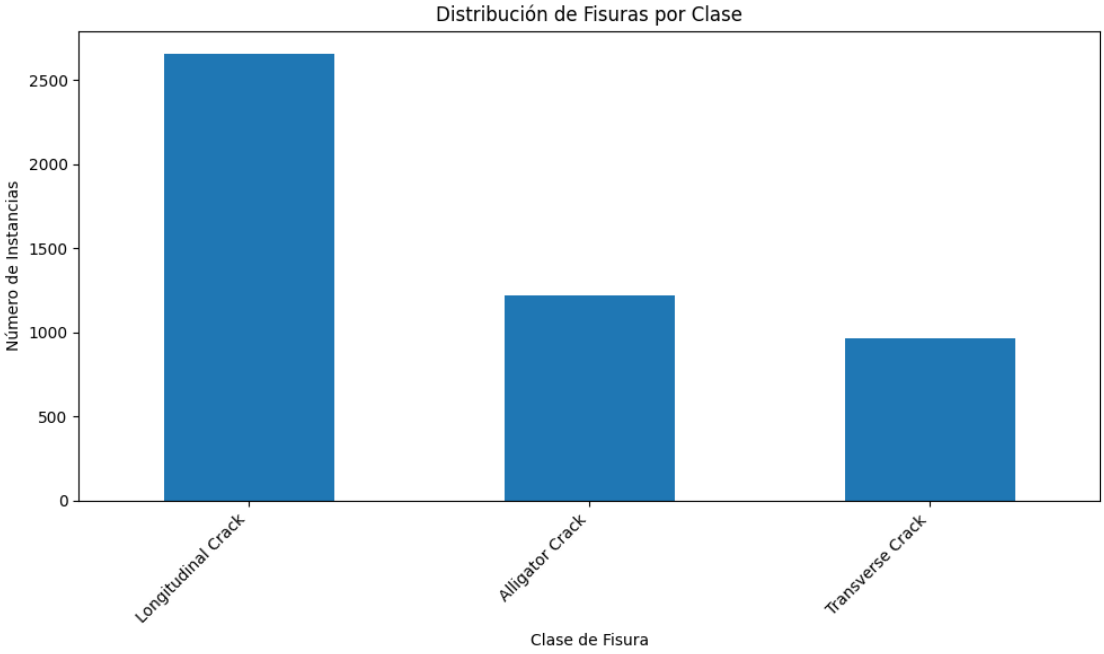


Figura 1. Distribución de instancias de fisuras por clase.



Figura 2. Ejemplos de imágenes originales con bounding boxes y etiquetas.

Observación: La distribución presenta desequilibrio, la clase longitudinal representa más de la mitad de las instancias, mientras que la transversal es la menos frecuente.

6. Conversión y preparación del dataset

Las anotaciones YOLO describen la posición de cada daño mediante coordenadas normalizadas. Se convirtió esas coordenadas a píxeles, validó sus límites y recortó la región correspondiente de la imagen original. Cada recorte fue guardado en una carpeta asociada a su clase.

6.1 Flujo de procesamiento

1. Recorrer las imágenes disponibles en el directorio del dataset.
2. Buscar y leer el archivo TXT de anotación correspondiente.
3. Convertir las coordenadas YOLO normalizadas a coordenadas absolutas.
4. Validar los límites del bounding box para evitar posiciones fuera de la imagen.
5. Recortar cada región anotada.
6. Descartar recortes vacíos o imágenes que no puedan cargarse.
7. Guardar los recortes en carpetas separadas por clase.
8. Registrar la ruta del recorte y su etiqueta en un DataFrame.

El proceso produjo 4.843 imágenes recortadas. Este cambio es central: el modelo se entrena sobre regiones que contienen una fisura concreta, no sobre la imagen vial completa.

6.2 División estratificada

La división se realizó con una semilla fija de 42 y estratificación por clase. La estratificación mantiene aproximadamente la misma proporción de categorías en los tres subconjuntos y mejora la comparabilidad de los resultados.

Conjunto	Proporción	Total	Longitudinal	Transversal / Alligator
Entrenamiento	70 %	3.390	1.860	675 / 855
Validación	15 %	726	398	145 / 183
Prueba	15 %	727	399	144 / 184

Los DataFrames resultantes fueron guardados como archivos CSV para conservar las rutas y etiquetas de cada partición.

7. Preprocesamiento y aumento de datos

7.1 Redimensionamiento y tamaño de lote

Todas las imágenes se adaptaron a 224 × 224 píxeles con tres canales RGB, tamaño esperado por MobileNetV2. El entrenamiento utilizó lotes de 32 imágenes. El procesamiento por lotes permite aprovechar mejor la aceleración disponible y estabilizar el cálculo de gradientes.

7.2 Normalización

Las imágenes fueron normalizadas con las medias 0,485, 0,456 y 0,406 y las desviaciones estándar 0,229, 0,224 y 0,225. Estos valores corresponden al esquema utilizado habitualmente con modelos preentrenados en ImageNet y permiten presentar los datos en una escala compatible con el conocimiento transferido.

7.3 Data Augmentation

El aumento de datos se aplicó únicamente al conjunto de entrenamiento mediante Albumentations. Las transformaciones simulan variaciones que pueden aparecer en condiciones reales y reducen la probabilidad de que el modelo memorice las imágenes originales.

Transformación	Configuración
Volteo horizontal	Probabilidad 50 %.
Desplazamiento, escala y rotación	Hasta 5 % de desplazamiento, ±5 % de escala y ±15° de rotación; probabilidad 75 %.
Brillo y contraste	Variación de hasta ±20 %; probabilidad 75 %.
Ruido gaussiano	Probabilidad 20 %.
Desenfoque de movimiento	Probabilidad 20 %.
Redimensionamiento	Conversión final a 224 × 224.
Normalización	Parámetros compatibles con ImageNet.

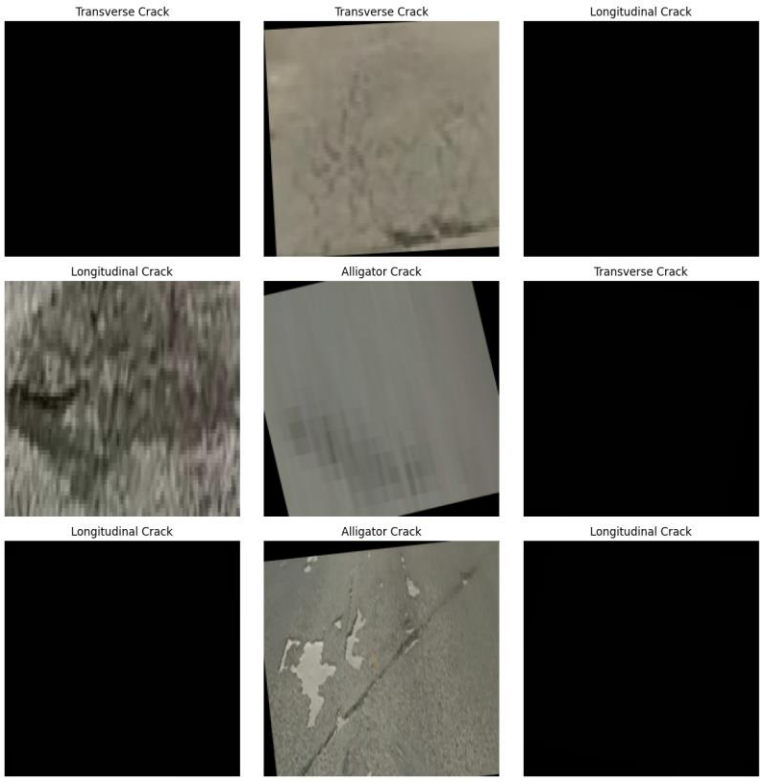


Figura 3. Ejemplos de imágenes originales de la cuadrícula

Visualización: Algunas de las figuras de la reconstrucción visual de imágenes normalizadas aparece oscurecida, aunque las transformaciones sí fueron aplicadas durante el entrenamiento.

7.4 Generadores de datos

Se construyeron generadores para leer las imágenes desde las rutas registradas en los DataFrames. El generador de entrenamiento aplica transformaciones aleatorias; los de validación y prueba realizan únicamente redimensionamiento y normalización. Las etiquetas se manejan en formato categórico one-hot.

Los generadores validaron 3.390 imágenes de entrenamiento, 726 de validación y 727 de prueba. El orden interno de clases fue: Alligator Crack = 0, Longitudinal Crack = 1 y Transverse Crack = 2.

8. Arquitectura del modelo

El modelo se construyó con la API funcional de Keras. MobileNetV2 fue cargada con pesos de ImageNet y sin su clasificador original. Sobre la salida convolucional se incorporó una cabeza de clasificación adaptada a las tres categorías del proyecto.

Componente	Función
Entrada	Imagen RGB de 224 × 224 × 3.
MobileNetV2 sin capa superior	Extractor de características; salida 7 × 7 × 1.280.

GlobalAveragePooling2D	Resume cada mapa de características en un valor y genera un vector de 1.280 elementos.
Dropout	Tasa 0,2 para regularización.
Dense	128 neuronas con activación ReLU.
Dropout	Segunda regularización con tasa 0,2.
Salida	3 neuronas con softmax para producir probabilidades por clase.

En la primera fase el modelo tuvo 2.422.339 parámetros totales. De ellos, 164.355 fueron entrenables y 2.257.984 permanecieron congelados. Esta configuración concentró el aprendizaje inicial en la nueva cabeza de clasificación, preservando las características generales aprendidas por MobileNetV2.

9. Compilación y estrategia de entrenamiento

9.1 Compilación inicial

- Optimizador Adam con tasa de aprendizaje de 0,0001.
- Función de pérdida categorical crossentropy, apropiada para clasificación multiclase con etiquetas one-hot.
- Métrica principal: accuracy.

9.2 Callbacks

Se configuraron mecanismos automáticos para controlar la convergencia y conservar la mejor versión del modelo:

- EarlyStopping: monitoreo de val_accuracy, paciencia de 10 épocas y restauración de los mejores pesos.
- ReduceLROnPlateau: reducción de la tasa de aprendizaje a la mitad después de 5 épocas sin mejora, con mínimo de 0,000001.
- ModelCheckpoint: almacenamiento del modelo con mejor val_accuracy en best_model.h5.

9.3 Entrenamiento inicial

La primera fase se ejecutó durante 20 épocas con la base MobileNetV2 congelada. La accuracy de validación pasó de 73,69 % en la primera época a 87,88 % en la época 20. El mejor valor guardado en esta fase fue precisamente 87,88 %, con una pérdida de validación de 0,3462.

La accuracy mostrada para entrenamiento fue considerablemente menor que la de validación. Esto puede explicarse por el fuerte aumento de datos aplicado durante el entrenamiento, que introduce ruido, rotaciones, desenfoques y variaciones de iluminación. La validación, en cambio, se realiza sobre imágenes normalizadas sin transformaciones aleatorias y representa una tarea más estable.

10. Fine tuning

Después de entrenar la cabeza de clasificación se habilitó el entrenamiento de MobileNetV2 y se mantuvieron congeladas las primeras 100 capas. El objetivo fue adaptar las capas superiores, más

especializadas, a los patrones particulares de las fisuras sin modificar excesivamente las representaciones generales de las capas iniciales.

El modelo fue recompilado con Adam y una tasa reducida de 0,00001. El fine tuning se planificó para 10 épocas adicionales, con callbacks más sensibles: paciencia de 5 épocas para EarlyStopping, reducción de learning rate después de 3 épocas sin mejora y guardado en best_fine_tuned_model.h5.

El mejor valor de validación de esta fase fue 86,91 % al comenzar el ajuste. Posteriormente no se produjo una mejora y la tasa se redujo a aproximadamente 0,000005. El entrenamiento se detuvo anticipadamente en la época 25 debido al criterio de *Early Stopping*. Esto indica que el ajuste fino (*fine-tuning*) no logró superar el mejor desempeño obtenido durante la primera fase de entrenamiento. No obstante, para la evaluación final se utilizó automáticamente el modelo con mejor rendimiento registrado durante todo el proceso de entrenamiento, garantizando que los resultados correspondieran a la versión con mayor capacidad de generalización.

Interpretación: El fine tuning no garantizó una mejora de la accuracy de validación. Su aporte debe evaluarse sobre datos de prueba y no solo por la cantidad de capas descongeladas.

11. CNN desde cero

La CNN de referencia se inicializa sin pesos preentrenados. Se compone de cuatro bloques Conv2D con 32, 64, 128 y 256 filtros de 3 × 3, ReLU y padding="same". Cada bloque incluye BatchNormalization y MaxPooling2D de 2 × 2. La resolución espacial disminuye progresivamente de 224 a 112, 56, 28 y 14 píxeles por lado.

La cabeza contiene GlobalAveragePooling2D, Dense de 128 unidades con ReLU, Dropout de 0,50 y Dense de tres unidades con softmax. El preprocesamiento específico escala a [0,1]. Se compila con Adam a 0,001 y categorical_crossentropy, y se entrena durante 50 épocas con los mismos tamaños de partición. Se fijan semillas de TensorFlow, NumPy y random a 42, aunque esto no controla necesariamente toda la aleatoriedad de Albumentations ni las operaciones del entorno.

Los callbacks monitorean val_accuracy con paciencia 10 para detención y 5 para reducir la tasa; el checkpoint es best_custom_cnn_model.h5. La mejor accuracy de validación alcanza 0,8168 en la época 46. El entrenamiento llega a las 50 épocas y la evaluación recarga ese checkpoint. En prueba se obtiene accuracy 0,799175 y loss 0,529522, con 581 aciertos sobre 727 ejemplos.

11.1 Motivos de fluctuación en entrenamiento y validación

Las salidas muestran oscilaciones importantes: la accuracy de validación es 0,8168 en la época 46, 0,2466 en la 49 y 0,7590 en la 50. En cambio, la accuracy de entrenamiento permanece cerca de 0,58 en el tramo final. La pérdida de validación sube a 3,8183 en la época 49. El cambio de una partición fija entre épocas refleja cambios del modelo, no un nuevo muestreo de la validación.

La optimización por minibatches y el aumento aleatorio introducen variabilidad. Dropout y perturbaciones hacen más exigente la medición de entrenamiento, de modo que validación puede superarla. El desbalance favorece estados que predicen principalmente la clase longitudinal, cuya proporción de validación es 398/726 = 54,82 %. Los registros cercanos a ese valor son compatibles con ese comportamiento, pero no lo confirman sin predicciones por época.

BatchNormalization usa estadísticas del lote al entrenar y medias móviles al validar. Una discrepancia entre imágenes aumentadas y originales puede afectar la estabilidad de evaluación. La tasa inicial de 10^{-3} y el aprendizaje de todos los filtros desde cero también pueden contribuir a ajustes más inestables. Estas son hipótesis técnicas, no causas aisladas mediante experimentos controlados.

Además, conviene auditar el tipo y rango de entrada a Albumentations: Keras entrega arrays float32 con escala de píxeles 0–255, mientras ciertas transformaciones distinguen entre uint8 y floats normalizados. El notebook no registra una validación sistemática de esta compatibilidad. Para investigar las fluctuaciones se propone verificar rangos, moderar augmentations, evaluar entrenamiento sin aumentos y comparar variantes con BatchNormalization congelada o una tasa inicial menor.

12. Evaluación del modelo

La evaluación final se realizó sobre 727 imágenes de prueba que no participaron en el entrenamiento ni en la validación. Se cargó el mejor modelo almacenado después del fine tuning y se calcularon pérdida, accuracy, reporte por clase y matriz de confusión.

12.1 Resultados globales

Modelo	Accuracy	Loss	Aciertos
MobileNetV2 congelada	84,73 %	0,398720	616/727
MobileNetV2 con fine tuning	84,59 %	0,384441	615/727
CNN desde cero	79,92 %	0,529522	581/727

MobileNetV2 congelada supera a la CNN en 4,81 puntos porcentuales; la variante ajustada, en 4,68 puntos. Las diferencias describen esta ejecución. No se realizaron múltiples semillas, intervalos de confianza ni una prueba estadística pareada. Tampoco se trata de una ablación exclusiva del preentrenamiento: cambian arquitectura, regularización, normalización, tasa de aprendizaje y presupuesto de épocas.

12.2 Resultados por clase

Clase	Precisión	Recall	F1-score	Soporte
MobileNetV2 FT · Cocodrilo	0,88	0,83	0,86	184
MobileNetV2 FT · Longitudinal	0,83	0,94	0,88	399
MobileNetV2 FT · Transversal	0,87	0,61	0,72	144
CNN · Cocodrilo	0,72	0,86	0,79	184
CNN · Longitudinal	0,82	0,87	0,85	399
CNN · Transversal	0,87	0,51	0,65	144

Precision indica la proporción de aciertos entre las predicciones de una clase; recall, la proporción de sus ejemplos reales recuperados; F1 combina ambas. Se reproducen los valores redondeados a dos decimales del notebook. MobileNetV2 ajustada alcanza F1 macro 0,82 y ponderado 0,84; la CNN, 0,76 y 0,79 respectivamente.

La clase transversal presenta el menor recall: MobileNetV2 identifica 88 de 144 ejemplos y la CNN 74 de 144. Aunque su precision es alta, quedan numerosos casos sin reconocer como transversales. La clase longitudinal obtiene mayor recall y es la mayoritaria. La asimetría destaca por qué la accuracy global resulta insuficiente para valorar una herramienta de inspección.
El notebook no incluye un reporte por clase para la variante congelada, por lo que no se atribuyen métricas específicas a ese modelo. Su comparación se limita a loss y accuracy registradas en la tabla final.

12.3 Matriz de confusión

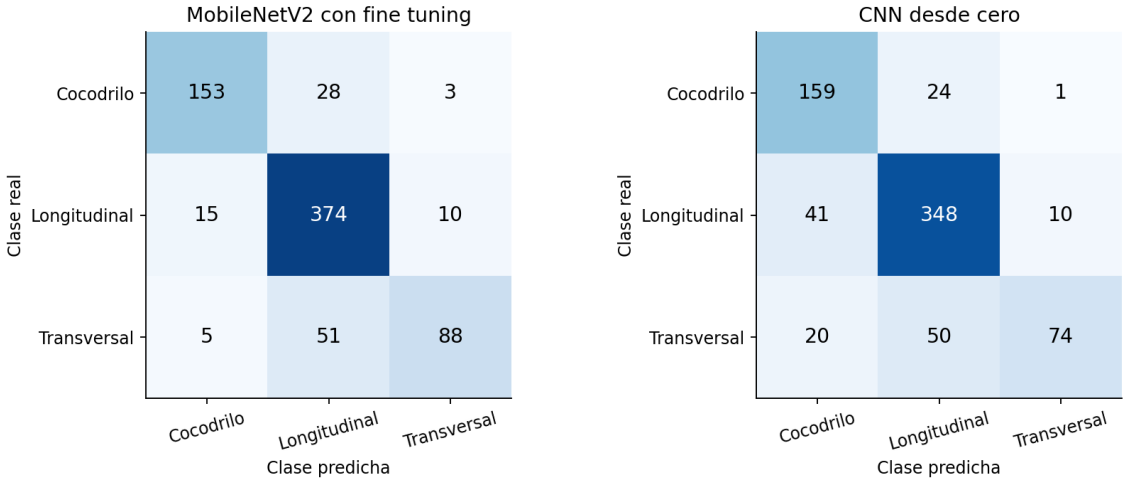


Figura 4. Matriz de confusión obtenida sobre el conjunto de prueba.

En MobileNetV2 ajustada, la diagonal contiene 153, 374 y 88 aciertos para cocodrilo, longitudinal y transversal. El error más frecuente es transversal → longitudinal, con 51 casos; le sigue cocodrilo → longitudinal, con 28. En la CNN, 50 transversales se clasifican como longitudinales y 41 longitudinales como cocodrilo. La revisión visual de estos recortes permitiría distinguir similitud de patrones, falta de contexto y posibles errores de anotación.

12.4 Curvas de entrenamiento

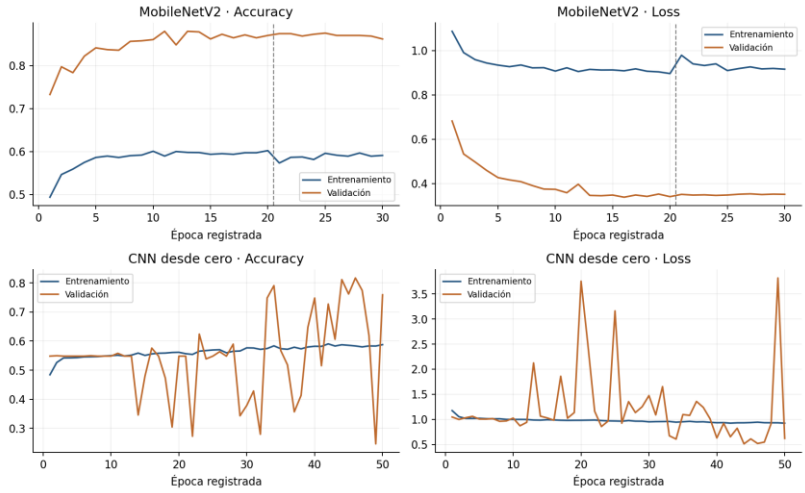


Figura 5. Curvas de accuracy y loss del entrenamiento.

MobileNetV2 mejora rápidamente en validación durante la fase congelada y luego se estabiliza. El ajuste fino no supera su máximo previo. La CNN muestra fuertes oscilaciones de validación y aprendizaje más lento. La brecha entre entrenamiento y validación debe interpretarse considerando augmentations,

Dropout y BatchNormalization, sin concluir automáticamente que existe sobreajuste por el solo orden de las curvas.

13. Predicción sobre imágenes nuevas

Se incorporó un flujo para cargar una imagen desde el equipo del usuario y clasificarla con el modelo final. La imagen se procesa con las mismas operaciones de validación y prueba: redimensionamiento a 224 × 224 y normalización.

El modelo devuelve una probabilidad para cada categoría mediante softmax. La clase con mayor probabilidad se presenta como resultado final. Además, el procedimiento mide el tiempo de inferencia en milisegundos y muestra la imagen original con la etiqueta predicha.

Este módulo constituye una demostración funcional del uso del modelo. Para una aplicación real debería integrarse con un sistema de captura, validación de imágenes, almacenamiento de resultados, georreferenciación y gestión de inspecciones.

14. Aplicación en empresas constructoras y viales

El sistema puede incorporarse como apoyo a la inspección y gestión del mantenimiento. No reemplaza por completo la evaluación profesional, pero puede acelerar la revisión de grandes volúmenes de imágenes y priorizar los casos que requieren intervención.

- Inspección automatizada: reducción del tiempo destinado a clasificar manualmente imágenes.
- Detección temprana: identificación de patrones de daño antes de que evolucionen.
- Mantenimiento proactivo: planificación de intervenciones según el tipo de fisura.
- Optimización de recursos: asignación de materiales, personal y equipos según el diagnóstico inicial.
- Consistencia: aplicación del mismo criterio de clasificación sobre todas las imágenes.
- Toma de decisiones basada en datos: generación de estadísticas históricas y análisis de tendencias.
- Seguridad vial: contribución a una reparación más oportuna de los daños.

15. Limitaciones identificadas

- Dependencia de los bounding boxes: un error en la anotación puede producir un recorte incompleto, irrelevante o con demasiado ruido.
- Generalización limitada: el rendimiento puede disminuir ante pavimentos, cámaras, resoluciones o iluminaciones diferentes a las del dataset.
- Desequilibrio de clases: la clase longitudinal tiene muchas más instancias que la transversal.
- Pérdida de contexto: al utilizar recortes, el modelo no observa el entorno completo de la fisura.
- Ausencia de clase negativa: el modelo siempre debe elegir una de las tres clases y no puede indicar “sin fisura”.
- Clasificación sin severidad: el sistema reconoce el tipo, pero no cuantifica longitud, ancho, área o nivel de daño.
- Fine tuning sin mejora clara: la segunda fase no superó el mejor resultado de validación obtenido durante el entrenamiento inicial.

16. Mejoras futuras

- Incorporar una clase “Sin fisuras” y ejemplos negativos diversos.
- Aplicar ponderación de clases, sobremuestreo o focal loss para reducir el impacto del desequilibrio.
- Revisar las transformaciones geométricas para evitar que la rotación vuelva ambiguas las clases longitudinal y transversal.
- Evaluar arquitecturas alternativas como EfficientNetB0 y comparar precisión, parámetros y tiempo de inferencia.
- Implementar detección directa con YOLOv5 o YOLOv8 para localizar y clasificar daños en una sola etapa.
- Utilizar segmentación semántica con U-Net para delinear la fisura píxel a píxel y medir su extensión.
- Entrenar con datasets más diversos que incluyan sombras, lluvia, suciedad, reparaciones y diferentes tipos de asfalto.
- Realizar validación externa con imágenes de obras reales no pertenecientes al dataset.
- Optimizar el modelo para TensorFlow Lite y desplegarlo en dispositivos móviles, cámaras o drones.
- Integrar geolocalización, fecha, tramo vial y nivel de prioridad en un sistema de mantenimiento.

17. Conclusiones

El proyecto demostró que es viable construir un clasificador automático de fisuras en pavimentos mediante MobileNetV2 y Transfer Learning. La preparación de los datos permitió convertir 3.321 imágenes anotadas para detección en 4.843 recortes aptos para clasificación. El uso de una partición estratificada, aumento de datos, callbacks y una arquitectura preentrenada conformó un pipeline completo y reproducible dentro de Google Colab.

El modelo alcanzó una accuracy final de 84,87 %, un resultado sólido para una primera implementación con tres clases. La clasificación fue especialmente efectiva para fisuras longitudinales y tipo piel de cocodrilo. El análisis por clase permitió detectar una debilidad específica en el recall de las fisuras transversales, lo cual orienta claramente las próximas mejoras.

Desde una perspectiva empresarial, el modelo puede funcionar como componente de apoyo para la inspección vial, la priorización de mantenimiento y la generación de información objetiva. Para avanzar hacia un producto operativo se requiere ampliar la diversidad del dataset, incorporar imágenes sin daño, detectar fisuras directamente en escenas completas y validar el sistema con datos reales de la organización.

En síntesis, el TP no se limita al entrenamiento de una red neuronal: abarca la carga y exploración de datos, conversión de anotaciones, preprocesamiento, aumento, diseño del modelo, entrenamiento en dos etapas, evaluación detallada y preparación de un mecanismo de inferencia. Esto constituye una base sólida para futuras soluciones de mantenimiento vial asistidas por Inteligencia Artificial.

18. Síntesis técnica del experimento

Elemento	Configuración / resultado
Dataset	Road Damage Dataset (CC0) - Kaggle

Elemento	Configuración / resultado
Imágenes originales	3.321
Instancias recortadas	4.843
Clases	Longitudinal, Transverse y Alligator Crack
Partición	70 % entrenamiento, 15 % validación y 15 % prueba
Entrenamiento / validación / prueba	3.390 / 726 / 727
Entrada	224 × 224 × 3
Batch size	32
Modelo base	MobileNetV2 con pesos ImageNet, sin clasificador superior
Cabeza personalizada	GlobalAveragePooling2D, Dropout 0,2, Dense 128 ReLU, Dropout 0,2, Dense 3 Softmax
Parámetros iniciales	2.422.339 totales; 164.355 entrenables
Optimizador inicial	Adam, learning rate 0,0001
Pérdida	Categorical crossentropy
Entrenamiento inicial	20 épocas
Fine tuning	Capas superiores; learning rate 0,00001; detención anticipada
Accuracy de prueba	84,87 %
Loss de prueba	0,3764